Univerzita Pardubice. Fakulta Elektrotechniky a Informatiky

Semestrální práce

CourtSy – webová aplikace sportovního klubu

Frontendová SPA aplikace s využitím technologie React

Vypracoval: Daniel Svoboda

Předmět: KTWA2

Vyučující: Ing. Dominik Štursa, Ph.D.

Studijní obor: Webové technologie

Akademický rok: 2026/27

V Třešti dne 23.10.2026

Obsah

Úvod ... 3

Zadání semestrální práce ... 4

1 Popis aplikace ... 5

1.1 Účel aplikace ... 5

1.2 Přehled obrazovek ... 5

2 Framework a struktura projektu ... 7

2.1 Zvolené technologie ... 7

2.2 Struktura projektu ... 7

2.3 Spuštění projektu ... 8

3 Vizuální styl a design systém ... 9

3.1 Barevná paleta ... 9

3.2 Typografie a rozvržení ... 9

3.3 Prvky rozhraní ... 10

4 Architektura komponent a routování ... 11

4.1 Vrstvy komponent ... 11

4.2 Routování ... 11

4.3 Vlastní ovládací prvky a animace ... 11

5 Správa stavu a práce s daty ... 12

5.1 Trvalý stav a LocalStorage ... 12

5.2 Napojení stavu na React ... 12

5.3 Mazání dat a odolnost úložiště ... 12

6 Validace vstupů a zabezpečení ... 13

6.1 Validační pravidla ... 13

6.2 Bezpečnostní opatření ... 13

7 SEO ... 14

7.1 Metadata stránek ... 14

7.2 Indexace a obrázky ... 14

8 Testování a optimalizace ... 15

8.1 Automatické testy ... 15

8.2 Profilování a optimalizace ... 15

9 Splnění požadavků zadání ... 16

10 Závěr ... 17

Úvod

Tato dokumentace popisuje frontendovou webovou aplikaci CourtSy, kterou jsem vytvořil jako semestrální práci k předmětu KTWA2. Jde o jednostránkovou aplikaci (SPA) fiktivního soukromého sportovního klubu s tenisovými kurty, golfovým hřištěm, posilovnou, regeneračním centrem a krytým bazénem. Návštěvník si v ní porovná úrovně členství, projde areály a tréninkové programy, prohlédne si kalendář akcí, nakoupí v klubovém e-shopu, projde objednávkovým formulářem a v uživatelském účtu spravuje svůj profil, nastavení a historii objednávek.

Aplikace navazuje na můj dřívější rozpracovaný projekt. Původně šlo o jednu dlouhou stránku s doplňkovými statickými podstránkami v HTML. Pro účely semestrální práce jsem ji přepracoval na plnohodnotnou SPA s routováním, centrální správou stavu, validací vstupů, péčí o SEO, automatickými testy a měřením výkonu. Zdrojový kód je veřejně v repozitáři a nasazená aplikace běží na GitHub Pages.

Dokument je členěn podle požadavků zadání. Nejprve popisuje aplikaci a její obrazovky, poté zvolený framework a strukturu projektu, vizuální styl, architekturu komponent, správu stavu a dat, validaci vstupů, SEO, a nakonec testování a optimalizaci. Poslední kapitola shrnuje, jak jednotlivé požadavky zadání aplikace splňuje.

Zadání semestrální práce

Cílem semestrální práce je navrhnout a implementovat frontendovou část webové aplikace v souladu s moderními zásadami vývoje webových technologií. Backend část není předmětem hodnocení, data lze simulovat pomocí mockovaných dat nebo úložiště na straně klienta. Aplikace musí splňovat tyto požadavky:

- SPA řešení s komponentovým přístupem (např. React, Vue) a implementací routování;
- správa stavu aplikace a práce s daty na straně klienta (LocalStorage, IndexedDB, mockovaná API);
- základní prvky zabezpečení a validace vstupů;
- minimálně pět logicky odlišných zobrazení (view), každé s vlastní komponentou a dostupné přes navigaci;
- ukázka testování (jednotkové nebo e2e testy) a profilování a optimalizace výkonu;
- stručná technická dokumentace popisující framework, strukturu projektu, architekturu komponent, správu stavu a dat a použité metody testování.

Projekt se odevzdává jako zdrojový kód v repozitáři, jehož součástí musí být README s instrukcemi pro spuštění, zdrojový kód aplikace, dokumentace a testy.

1 Popis aplikace

1.1 Účel aplikace

Projekt simuluje webovou prezentaci a prodej členství soukromého sportovního klubu. Návštěvník má možnost na webu:

- porovnat pět úrovní členství od jednodenní vstupenky po firemní tarif a prohlédnout si detail každé z nich;
- projít pět areálů klubu a rezervovat si konkrétní termín, například hodinu na centrálním kurtu;
- projít devět tréninkových programů, každý s rozpisem bloku a trenéry, kteří ho vedou;
- prohlédnout kalendář klubových akcí (turnaje, ligy, společenské večery aj.) a přihlásit se na ně;
- nakoupit v klubovém e-shopu vybavení a doplňky a podívat se do klubové restaurace;
- zapsat se na prohlídku klubu, zjistit podmínky studentské slevy nebo koupit dárkový poukaz;
- projít procesem objednávky s kompletní validací vstupů;
- spravovat svůj profil, nastavení a historii objednávek, případně svůj účet celý smazat.

Aplikace nemá žádný server. Obsah webu (členství, areály, programy, akce, zboží) je uložen jako statická data ve složce src/data a vše, co vytváří uživatel - košík, profil, objednávky a nastavení - se ukládá do localStorage prohlížeče. Žádná data se nikam neodesílají, platba je pouze simulovaná.

1.2 Přehled obrazovek

Aplikace má 21 různých směrování, která obsluhují 19 komponent obrazovek ve složce src/views. Každá obrazovka má vlastní URL, takže na ni lze přejít z navigace i přímým odkazem. Tři právní stránky sdílejí jednu komponentu Legal, protože mají stejnou osnovu a liší se jen textem.

Cesta (URL)	Komponenta	Obsah obrazovky
/	Home	úvodní stránka s postupně odkrývanými sekcemi (FadeIn effect)
/memberships	Memberships	přehled pěti úrovní členství, filtrování a řazení
/memberships/:slug	MembershipDetail	detail úrovně členství, přidání do košíku
/courts	Courts	přehled areálů s vyhledáváním
/courts/:slug	CourtDetail	detail areálu, galerie, rezervace termínu

Cesta (URL)	Komponenta	Obsah obrazovky
/programmes	Programmes	přehled tréninkových programů a časová osa trenérů
/programmes/:slug	ProgrammeDetail	detail programu: rozpis bloku, fakta, trenéři
/events	Events	kalendář akcí, filtr, hledání, seskupení po měsících
/events/:slug	EventDetail	detail akce a registrace
/shop	Shop	klubový e-shop s filtrováním podle kategorie
/dining	Dining	klubová restaurace: tři sály, otevírací doba a menu
/join	Join	co členství obnáší, ještě než dojde na ceník
/private-tour	PrivateTour	termíny prohlídek, zápis jménem bez platby
/students	Students	studentská sleva, sjednává se osobně (bez nákupu)
/gift	Gift	dárkové poukazy ve čtyřech hodnotách
/checkout	Checkout	čtyřkrokový objednávkový formulář s validací
/account	Account	profil, nastavení, historie objednávek, smazání účtu
/terms, /privacy, /cookies	Legal	obchodní podmínky, ochrana soukromí, cookies
jiná adresa	NotFound	stránka 404

Tabulka 1 Přehled obrazovek aplikace

2 Framework a struktura projektu

2.1 Zvolené technologie

Aplikace je postavena na knihovně **React** a nástroji **Vite**. React jsem zvolil kvůli komponentovému modelu, se kterým jsem měl zkušenosti z původního projektu, a kvůli hooku useSyncExternalStore, který řeší napojení Reactu na state uložený mimo něj. Vite poskytuje rychlý vývojový server a bez další konfigurace dělí výsledný kód do samostatných souborů podle směrování. V projektu jsou použity tyto technologie:

- **React**, verze 18.3 (komponentová architektura uživatelského rozhraní);
- **React Router**, verze 6.30 (routování v režimu data routeru, načítání obrazovek až při potřebě);
- **Vite**, verze 5.4 (vývojový server, produkční build, dělení kódu, generování mapy webu);
- **Framer Motion**, verze 11 (animace a efekty řízené scrollem, respektování omezeného pohybu);
- **Inter** a **Aspekta** (písma hostovaná přímo v aplikaci, viz kapitola 3);
- **Vitest**, verze 2.1 (spouštění jednotkových a komponentových testů);
- **React Testing Library**, verze 16 (testování komponent z pohledu uživatele);
- **Playwright**, verze 1.64 (end-to-end testy ve skutečném prohlížeči Chromium);
- **Lighthouse**, verze 13 (měření výkonu, přístupnosti, osvědčených postupů a SEO);
- **Sharp**, verze 0.35 (zmenšení fotografií a převod do formátu WebP);
- **GitHub Actions** a **GitHub Pages** (automatické testy, sestavení a nasazení po každé změně).

TypeScript jsem nepoužil. Zadání ho doporučuje, ale nevyžaduje, a převod existujícího JavaScriptového projektu by k funkčnosti nic nepřidal. Datové struktury a veřejné funkce jsou místo toho popsané typovými komentáři JSDoc (např. @typedef Tier, ClubEvent, Court), díky kterým editor nabízí stejné našeptávání a kontrolu typů.

2.2 Struktura projektu

Zdrojový kód je členěn podle role souborů, ne podle jejich typu:

- src/main.jsx – vstupní bod aplikace;
- src/App.jsx a src/routes.js – tabulka směrování a jediné místo, kde jsou definované URL adresy;
- src/layouts/RootLayout.jsx – společný rám aplikace (navigace, košík, patička);
- src/views/ – obrazovky, každá jako samostatná komponenta;
- src/components/ – sdílené komponenty rozdělené na layout/, sections/, commerce/ a ui/;
- src/context/StoreContext.jsx – React kontext zpřístupňující stav aplikace;

- src/lib/ – logika bez závislosti na Reactu: store.js, validation.js, format.js, seo.js, scroll.js;
- src/data/ – veškerý obsah webu (mockovaná data);
- src/styles/ – návrhové tokeny v index.css a styly rozdělené podle sekcí;
- tests/unit/ a tests/e2e/ – jednotkové, komponentové a end-to-end testy;
- docs/ a scripts/ – výsledky měření Lighthouse a pomocné skripty (obrázky, audit, kontrola rout).

Oddělení složky lib/ je záměrné. Nic v ní neimportuje React, takže logiku stavu, validace a SEO lze testovat samostatně bez vykreslování komponent.

2.3 Spuštění projektu

Projekt vyžaduje Node.js ve verzi 22 nebo novější. Postup spuštění je popsán také v souboru README:

- npm install – instalace závislostí;
- npm run dev – vývojový server na adrese http://localhost:5173;
- npm run build a npm run preview – produkční build a jeho náhled;
- npm test – jednotkové a komponentové testy (Vitest);
- npm run test:e2e – end-to-end testy (Playwright), build a server si spustí samy;
- npm run audit – měření Lighthouse nad spuštěným náhledem.

Po každém pushi do branch main spustí GitHub Actions nejprve všechny testy, poté sestaví produkční build a publikuje ho na GitHub Pages. Pokud některý test neprojde, nasazení se neprovede.

3 Vizuální styl a design systém

Vzhled aplikace stojí na sadě návrhových tokenů – CSS proměnných definovaných jednou v src/styles/index.css na prvku :root. Barvy, písma, zaoblení, odsazení i křivky animací se ve stylech nezadávají přímo, ale odkazem na token (např. var(--ink)). Změna tokenu se tak projeví v celé aplikaci najednou. Styl vychází z minimalismu.

3.1 Barevná paleta

Paleta není schválně příliš rozsáhlá. Text je vždy téměř černý, volné plochy jsou bílé a světle šedé a zelená se používá jen pro akce, štítky a zvýraznění (např. tlačítka).

Ukázka	Token	Hodnota	Použití
	--ink	#0F0F0F	veškerý text, tmavá tlačítka, tmavé pruhy
	--ink-soft	#202020	tmavé tlačítko po najetí myší
	--paper	#FFFFFF	karty, formuláře, světlá tlačítka
	--page	#F8F8F8	pozadí stránky, pozadí vstupních polí
	--bone	#EDEDED	jemně odlišené plochy a sekce
	--mist	#DEDEDF	neutrální výplně a oddělovače
	--lime	#5DD62C	hlavní akcentní barva: tlačítka, štítky, šipky
	--lime-soft	#E4F8D8	světlé zelené podklady a odznaky
	--green	#337418	zelený text na bílé (čitelný kontrast), fokus pole
	–	#C7352B	chybové stavy formulářových polí

Tabulka 2 Barevná paleta aplikace

Oddělovací linky jsou poloprůhledně černé (rgba(15, 15, 15, 0.1) a 0.16), takže fungují na bílém i šedém podkladu.

3.2 Typografie a rozvržení

Aplikace používá dvě písma, obě hostovaná přímo v aplikaci, takže se nenačítají z cizí domény. Na velké nadpisy a čísla je použito **Inter**, které ve velkých velikostech působí pevně a geometricky. Základní text, popisky, tlačítka a formuláře jsou psané písmem **Aspekta**. Velikosti nadpisů se pomocí funkce clamp() mění plynule podle šířky obrazovky, takže na mobilu i velkém monitoru zachovávají stejné proporce.

Rozvržení vychází ze stejných pravidel na všech stránkách:

- obsah je vycentrovaný v kontejneru o maximální šířce 1440 px, s bočním odsazením, které roste od mobilu k velké obrazovce;
- sekce mají velkorysé svislé odsazení, takže stránka působí vzdušně;
- mezery mezi prvky řeší flexbox a grid pomocí vlastnosti gap, ne okraje jednotlivých prvků;

- pod šířkou zhruba 900 px se mřížky skládají do jednoho sloupce a navigace se mění na vysouvací menu;
- karty, tlačítka a štítky mají výrazně zaoblené rohy a jemné, rozptýlené stíny místo ostrých obrysů.

3.3 Prvky rozhraní

Tlačítka mají jednotný tvar pilulky a stejnou velikost písma i odsazení. Liší se jen barvou podle důležitosti akce: černé pro hlavní akci, bílé s obrysem pro vedlejší a zelené pro nejvýraznější výzvu. Po najetí myší se tlačítko mírně zvedne a dostane stín, při stisknutí se lehce zmenší.

Formulářová pole mají šedý podklad a tenký obrys. Při zaměření obrys zezelená, při chybě pole zčervená a pod ním se zobrazí hláška. Všechny animace používají jednu společnou křivku a respektují nastavení omezeného pohybu.

4 Architektura komponent a routování

4.1 Vrstvy komponent

Aplikace je poskládaná ve čtyřech vrstvách. Každá vrstva vykresluje tu následující:

1. Vstup a routování – main.jsx aplikaci spustí a obalí ji poskytovatelem stavu a routerem. App.jsx obsahuje tabulku všech směrování.
2. Rám aplikace – RootLayout obsahuje navigaci, košík, patičku a úvodní animaci. Připojí se jen jednou a při navigaci se nemění, mění se jen obsah uvnitř. Proto košík přežije přechod mezi stránkami.
3. Obrazovky – komponenty ve složce src/views, vždy je zobrazená jedna. Každá si bere svá data a drží vlastní lokální stav, například zvolený filtr.
4. Sdílené komponenty – opakující se stavební prvky, ze kterých se obrazovky skládají: hlavička stránky, formulářové pole, odkaz, responzivní obrázek, rozbalovací nabídka, potvrzovací dialog nebo galerie.

Obrazovky přitom pracují se čtyřmi moduly mimo React: čtou statická data ze src/data, stav získávají přes StoreContext (ten je napojený na lib/store.js, který zapisuje do localStorage), formuláře ověřují funkcemi z lib/validation.js a metadata stránky nastavují přes lib/seo.js.

4.2 Routování

Routování zajišťuje React Router 6 v režimu data routeru. Oproti jednoduchému <BrowserRouter> přináší obnovení pozice stránky při navigaci, chybovou hranici pro každou routu a načítání obrazovek až ve chvíli, kdy jsou potřeba. Úvodní stránka se načte hned, ostatní obrazovky jsou v samostatných souborech. Neznámá adresa zobrazí stránku 404.

Aplikace běží na GitHub Pages v podsložce /ModernityRay/. Build proto vytváří soubor 404.html jako kopii index.html. GitHub Pages ho vrátí pro každou neznámou adresu, načte se aplikace a ta si adresu zpracuje sama. Díky tomu fungují přímé odkazy na podstránky a adresy nepotřebují znak #.

4.3 Vlastní ovládací prvky a animace

Komponenty SelectMenu a ConfirmDialog nahrazují vestavěný <select> a window.confirm(), které nejdou sladit se vzhledem aplikace. Vrací přitom vše, co vestavěné prvky umí (ovládání klávesnicí, zavření klávesou ESC, návrat focusu a role pro čtečky obrazovky). Dialog má po otevření focus na tlačítku Zrušit.

Animace zajišťuje Framer Motion. Obrazovky se při přechodu plynule objeví, úvodní animace se zobrazí jen jednou za relaci a galerie areálu se sama posouvá. Při omezeném pohybu (v systému nebo v nastavení účtu) se animace i CSS přechody téměř vypnou.

5 Správa stavu a práce s daty

5.1 Trvalý stav a LocalStorage

Trvalý stav spravuje modul src/lib/store.js mimo React a ukládá ho do localStorage pod klíči s předponou:

Klíč	Co obsahuje
courtsy.cart	položky košíku (název, cena a počet kusů)
courtsy.user	jméno, e-mail a datum založení účtu, heslo se neukládá
courtsy.orders	posledních 20 objednávek (číslo, datum, položky, součty, kontakt a poslední čtyři číslice karty)
courtsy.prefs	nastavení (omezení animací, účtenky e-mailem, ceny včetně daně)
courtsy.schema	verze struktury uložených dat
courtsy.tours	zápisy na prohlídky, ukládá je přímo obrazovka PrivateTour, mimo store
courtsy.intro.seen	příznak zhlédnuté úvodní animace (sessionStorage, jen do zavření karty)

Tabulka 3 Data uložená v úložišti prohlížeče

5.2 Napojení stavu na React

Komponenty k uloženému stavu přistupují přes jediný hook useStore(). StoreContext je napojený na store pomocí useSyncExternalStore, hooku určeného právě pro stav uložený mimo React. Stav je díky tomu k dispozici už při prvním vykreslení.

Redux jsem nepoužil záměrně. Stav tvoří jeden košík, jeden profil a tři přepínače bez asynchronních operací, takže by knihovna přidala kód bez přínosu. Zadání vlastní řešení připouští a díky hooku useStore() by šla implementace kdykoli vyměnit. Dočasný stav (otevřený košík, krok formuláře, filtry) je v useState v komponentě.

5.3 Mazání dat a odolnost úložiště

Data lze smazat ve dvou úrovních. forgetMe() smaže profil, košík a objednávky, ale ponechá nastavení. deleteAccount() smaže i nastavení. Obě akce se nejprve ptají přes ConfirmDialog.

Práce s úložištěm je ošetřená pro běžné potíže:

- při spuštění se ověří, že úložiště funguje, a v anonymním režimu nebo při plné kvótě aplikace uživatele upozorní;
- každé čtení a zápis je v try/catch, takže chyba úložiště aplikaci neshodí;
- načtená data se znovu validují, poškozená položka košíku se zahodí, místo aby rozbila součty;
- klíče jsou verzované, takže po změně struktury se stará data smažou místo pádu aplikace.

6 Validace vstupů a zabezpečení

6.1 Validační pravidla

Validace je v modulu src/lib/validation.js jako sada čistých funkcí. Dostanou hodnotu a vrátí chybovou hlášku, nebo null, pokud je hodnota platná.

Pole	Pravidlo
E-mail	jeden znak @, tečkovaná doména, bez mezer, max. 160 znaků
Jméno	jméno a příjmení, pouze písmena, mezery, pomlčky a apostrofy
Heslo	min. 8 znaků, alespoň jedno písmeno a jedna číslice; neukládá se
Telefon	7–15 číslic
Adresa a PSČ	adresa s číslem domu, PSČ ve formátu zvolené země
Platební karta	Luhnův kontrolní součet, platnost v budoucnu, délka CVC podle vydavatele

Tabulka 4 Validační pravidla formulářových polí

Objednávka validuje každý krok zvlášť a pokračuje dál, až když jsou všechna pole platná. Neplatné pole se označí, chybová hláška se přečte čtečkou obrazovky a fokus skočí na první chybu.

6.2 Bezpečnostní opatření

- Žádná tajná data – heslo se po kontrole zahodí, z karty se uloží jen vydavatel a poslední čtyři číslice;
- Sanitizace – text se před uložením zkrátí a zbaví řídicích znaků, aby nešlo poškodit uložená data;
- Odmítnutí nemožných vstupů – neplatná karta nebo prošlá platnost se do stavu vůbec nedostanou;
- Ochrana proti XSS – aplikace nepoužívá dangerouslySetInnerHTML, React veškerý text escapuje sám.

7 SEO

7.1 Metadata stránek

SEO pomáhá vyhledávačům stránky správně zobrazit ve výsledcích a sociálním sítím vytvořit náhled odkazu. Jednostránková aplikace ale pro všechny adresy používá stejný index.html se stejným titulkem. Proto každá obrazovka volá hook useSeo(), který přes modul lib/seo.js nastaví vlastní titulek (např. Memberships | CourtSy), popis zkrácený na délku vhodnou pro výsledky vyhledávání, kanonickou adresu a náhled pro sociální sítě (Open Graph a Twitter Card).

7.2 Indexace a obrázky

- pokladna, účet a stránka 404 mají noindex a pokladna s účtem jsou zakázané i v robots.txt;
- mapa webu sitemap.xml se generuje automaticky při buildu z dat aplikace;
- index.html obsahuje strukturovaná data (JSON) popisující klub;
- alternativní texty obrázků jsou uložené na jednom místě u každé fotografie.

Kategorie SEO v Lighthouse dosahuje 100 bodů a čtyři end-to-end testy hlídají, že metadata na žádné stránce nechybí ani se neopakují.

8 Testování a optimalizace

8.1 Automatické testy

Projekt obsahuje 145 automatických testů. GitHub Actions je spouští před každým nasazením.

Vrstva	Nástroj	Počet	Co ověřuje
Jednotkové	Vitest	87	validace, store a výpočet součtů, formátování dat
Komponentové	Vitest + React Testing Library	15	formulářové pole, rozbalovací nabídka, průchod pokladnou
End-to-end	Playwright (Chromium)	43	navigace, nákupní cesta, nové stránky, SEO, úvodní animace

Tabulka 5 Přehled automatických testů

Jednotkové testy míří na logiku ve složce lib/, například Luhnův algoritmus nebo chování storu nad poškozeným úložištěm. End-to-end testy běží proti produkčnímu buildu a procházejí aplikaci jako návštěvník - celou nákupní cestu až po nalezení objednávky v účtu, filtry, přechody mezi stránkami nebo metadata. Jeden test navíc hlídá, že se při procházení všech stránek nikdy znovu nenačte celý dokument, tedy že aplikace zůstává opravdu jednostránková.

8.2 Profilování a optimalizace

Výkon byl měřen nástrojem Lighthouse nad produkčním buildem v emulaci mobilu. Reporty jsou ve složce docs/lighthouse/.

Route	Výkon	Přístupnost	LCP	CLS
/	70 → **88**	90 → **100**	9,8 s → **3,5 s**	0 → **0**
/memberships	68 → **92**	94 → **100**	3,2 s → **3,1 s**	0,58 → **0**
/courts/centre-court	57 → **89**	96 → **100**	5,0 s → **3,3 s**	0,59 → **0**

Tabulka 6 Výsledky měření Lighthouse před optimalizací a po ní

Best Practices i SEO mají po úpravách 100 bodů. Hlavní optimalizace:

- **posun rozvržení** – obrazovky se načítají přes lazy na úrovni route, takže se stránka vykreslí najednou a nic neposkakuje;
- **obrázky** – fotografie se převádějí do WebP ve čtyřech šířkách a prohlížeč si vybere vhodnou velikost. Hlavní obrázek se zmenšil z 293 kB na 72 kB;
- **písmo** – Inter je součástí buildu místo načítání z Google Fonts;
- **přístupnost** – opravený kontrast textů, pořadí nadpisů a velikost ovládacích prvků.

9 Splnění požadavků zadání

Požadavek	Řešení v projektu
SPA s komponentovým přístupem	React 18, komponenty ve složkách views/, components/, layouts/
Routing	React Router 6, načítání obrazovek až je potřeba, stránka 404
Správa stavu	vlastní store lib/store.js + StoreContext
Práce s daty na klientovi	localStorage a mockovaná data v src/data
Zabezpečení a validace	lib/validation.js – validace, sanitizace, neukládání citlivých údajů
Minimálně 5 zobrazení	21 směrování a 19 komponent obrazovek
Testování	102 testů ve Vitestu, 43 e2e testů v Playwrightu
Profilování a optimalizace	Lighthouse, WebP obrázky, dělení kódu
README a dokumentace	README.md s instrukcemi pro spuštění, tato dokumentace

Tabulka 7 Splnění požadavků zadání

10 Závěr

Cílem práce bylo vytvořit frontendovou část webové aplikace podle moderních zásad vývoje. Původní prezentační web jsem přepracoval na jednostránkovou aplikaci CourtSy s 21 směrováními, jednotným design systémem, správou stavu nad localStorage, validovaným objednávkovým formulářem a metadaty pro vyhledávače.

Kvalitu ověřuje 145 automatických testů a měření Lighthouse, podle kterého má výkon 88–92 bodů a přístupnost, Best Practices i SEO 100 bodů. Za hlavní přínos považuji oddělení logiky stavu, validace a SEO od Reactu, díky kterému je dobře testovatelná. Možným rozšířením je napojení na skutečné API, převod do TypeScriptu nebo ukládání dat do IndexedDB.

Seznam tabulek

Tabulka 1 Přehled obrazovek aplikace 6
Tabulka 2 Barevná paleta aplikace..... 9
Tabulka 3 Data uložená v úložišti prohlížeče..... 12
Tabulka 4 Validační pravidla formulářových polí 13
Tabulka 5 Přehled automatických testů 15
Tabulka 6 Výsledky měření Lighthouse před optimalizací a po ní..... 15
Tabulka 7 Splnění požadavků zadání..... 16